Bitwise ID + Set Management

A method to increase the detection speed of electronic sensor boards based on RFID

Version 1: February 16, 2016 Initial
Version 1.3: Feb 18 2016 + test results and error probability calculation
Version 1.4: Feb 22 2016. Addition of research into types of tags
Version 1.6: launched May 23, 2016. Added row and column encoding algorithm
Version 1.7: June 30, 2016: textual improvements
Version 1.8: November 16, 2016: Adding considerations Set management (appendix 3)
Version 1.9: March 3, 2017: Revision of text and addition of test system experiences
Version 2.0: Feb 09 2025: Manual corrections on auto translated text
Version 2.1: March 2026: small corrections

TABLE OF CONTENTS

Introduction

RFID object tracking and identification is an obvious and often used technology for the realization of electronic game boards (Chess boards, Go, Scrabble, planning boards, etc.). RFID tags are uniquely labeled. They may contain additional information about their meaning, and are available from mass production. The availability of RFID technology is guaranteed for a long time due to its popularity.

This document applies to “electronic game boards”, a definition of a device which has the following properties:
Objects can be placed, moved and/or removed by the user on a bounded two-dimensional surface (not necessarily flat). The objects are provided with one or more RFID tags on the contact surface with the playing surface. The electronic in-plane measurement system is able to determine the location and identity of the tags, and therefore of the objects. Using this functionality, a system can, among other things, record a game, and interactivity with the user can be achieved using other media (sound, light): tasks can be presented to the user and the execution can be monitored.

Typical applications and configurations are:
Chessboard: 8 x 8 squares, 34 chess pieces (extra set of Queens), 12 different identities. At the start of the game the largest number of pieces are on the field (32), during the game mainly decreasing in number. “Dead zones” (where an object cannot be detected) between the squares are allowed.
The system can record and publish a chess game played between two people or generate chess moves or analyze positions in interaction with the user.

Go board: 19 x 19 positions, theoretically 361 stones possible on the board, in practice about 320. Two identities (black, white). At the start of the game an empty board, during the game mainly increasing number of stones on the board. Discrete fields: dead zones allowed. The system can record and publish a Go game played between two people or generate Go moves in interaction with the user or perform analyzes on positions.

Scrabble board: 15x15 fields (positions), 100 tiles in a regular game, all of which can be on the board. In addition, 2 or more racks, on which up to 7 tiles can be placed. 27 or more identities (letters of the alphabet, blank, language-specific special letters). An empty board at the start of the game, with an increasing number of stones on the board during the game. Discrete fields: dead zones allowed. The system can record and publish a Scrabble game played between two or more people or check words placed in interaction with the user, keep score, make suggestions.

TagTile (TikTegel)[1] : 12 x 12 fields, limited number of tags on the board (1-12 pieces). Each tag has its own identity. The number of tags on the board is changing during a session. Continuous fields: no dead zones allowed.
The interaction can issue commands in interaction with the user and record the user's response in terms of, for example, reaction speed, correctness of executing the command.

[1] TikTegel or the English name TagTileis a concept, developed by serioustoys.com

For all game boards, the speed of detection of placing, moving or removing an object must be detected sufficiently quickly. Detection time under 1 second is acceptable, less than 0.5 second is desirable.

This report describes a method of how a sufficiently fast detection system can be realized for electronic game board systems with a large number of tags simultaneous on the board, using 13 MHz RFID tags [2]. Achieving the desired detection speed is not evident: Using conventional anticollision algorithms, the detection time increases more than proportionally to the number of tags on a board. In general, multiple detection systems have to work in parallel to solve this, causing an increase in product price and complexity.

At a glance: Solving the speed problem

The common method to read tags on a board is by having rectangular antennas under each row and each column of the board. For every antenna, anticollision reading algorithms need to be executed to make an inventory of all tags on the board.
The normal anticollision procedure is time consuming.
The proposed solution is to do block reads on all tags within the detection area of an antenna, so that all tags **simultaneously** send the content of a memory block. Normally this would lead to scrambling of the data, while they are all overlapping.
ICode tag communication works so, that the bits read by the reader are a logical-OR of all bits that are sent by the tags simultaneously. So, if we take care that never two logic-1 bits but only maximum of one logic 1 bit will be sent on the same time by the tags, never a collision will happen.
This allows to code the identification of a tag as a single 1-bit in a number of zero bits. I.e. for Scrabble, where 100 tiles can occur, a number of 100 bits of the tag memory are read, where each bit is assigned to each individual tile. By reading all 100 bits of all tiles, the tiles can be identified by each 1-bit in the data stream. This procedure is detailed in the following of this document.

Architecture of the measurement system

For this report it is assumed that the playing surface is equipped with several antennas, one antenna under each row and one antenna under each column of the board. These antennas can be alternately connected to an RFID reader, using an antenna multiplexing circuitry [3].
The antennas can be overlapping or non-overlapping. Tags that fall within the detection area of the antenna are recognized by that antenna, tags that fall outside that area are not recognized by that antenna [4].
The antennas can be overlapping, to avoid dead zones, or separated, creating dead zones between the fields. This report initially assumes non-overlapping antennas. An extension for overlapping antennas is added where necessary.
Depending on the application, such an antenna can contain 0 to 19 (for Go board) tags.

---

[2] The method has been tested with standard 15693 ICode tags. Under certain conditions, the method can be used for tags according to other standards, including 14443 Type A tags.

[3] A low cost multiplexing is implemented using PIN diodes which can switch between low resistance and high resistance with very low capacitance.

[4] More precisely, approximately 60-85% of the surface of a tag should fall within the antenna. This depends, among other things, on the height of the tag above the antenna surface.

The reading system reads all antennas sequentially and receives data from all tags that are within the reading range of the antenna. By combining the measurement data from all row and column antennas, the position of each tag can be determined.

This also applies in the case of overlapping antennas, with the special feature that tags within an overlap are detected by each of the overlapping antennas. Once again, the position of each tag can be determined by combining the measurement data from all antennas, whereby the overlap between antennas can even be regarded as an extra field.

In general, RFID tags have a unique serial number, the UID. In addition, they contain an amount of writeable and readable memory. The role of the object can be stored in this tag memory (Role examples: White pawn, Black stone, letter W with 8 points, red block, User smart card, Mickey Mouse).

Common , slow detection method

The usual detection method of multiple tags within an antenna is using standard anti-collision[5] techniques [1] [2].
The aim is to determine the UID of each tag present or to read part of the memory of each tag.

A measuring method could then be:

For all antennas , one by one, do:

Step 1: Detect whether 0, 1 or more tags are within an antenna using the Inventory (1 slot) command. This step can be skipped for antennas on which multiple tag are expected. In case of 1 tag found Continue step 3.
Step 2: If multiple tags detected (collision): Perform anti-collision cycle.
Step 3: The role is read for those tags for which the role is not yet known.
Step 4: Combine measurement data from all antennas and determine the position and role of all tags found. Communicate this with an application.

A calculation of the expected measurement times and the cycle time of the above method shows that this is longer than desired.

For the Chessboard situation:
There are 16 antennas (8 rows, 8 columns). There are 32 pieces on the board at the start of the game. There are 4 empty rows at the start of the game.

The time required for one complete scan at the initial setup is a sum of these measurements:
4 x anticollision procedure with 8 tags within an antenna (4 rows)
4 x an Inventory (1 slot) for 4 empty rows
8 x an anticollision procedure with 4 tags within an antenna (8 columns).

[5] Anticollision is a well-known procedure to prevent tags from communicating with a reader at the same time, which would disrupt communication. An anti-collision procedure ensures that all tags can be identified

The average (realizable) measurement time for these components is:
Inventory (1 slot) with 0 tags; 18 ms.
Inventory (1 slot) with 1 tag: 28 ms.
Inventory (1 slot) with multiple tags: 12.3 ms.
Anti-collision with 2 or more tags, without collision: 16.8 ms. + (number of tags*3.9)ms.
In the event of a collision, the Anticollision must be repeated with a mask value. Whether and how often a collision occurs can only be determined as an average by means of probability calculation [6].
Anticollision with 4 tags has a 33% chance that an anticollision will need to be performed again.
Anticollision with 8 tags results in an 88% chance that an anticollision will have to be performed again.
This results in a total time:
4 x 18 ms. for the empty rows: 72 ms. >> empty row also costs 18 ms!
12 x anticollision with an average of 4 responding tags (estimated): 12 x 39ms. = 396 ms.
6 to 10 anticollision with mask information and 4 response tags: 195 to 324 ms.
Total time: 616 to 745 ms. per measurement cycle.
With other configurations of the pieces, this time may even increase. As the number of pieces on a board decreases, the time for a measuring cycle generally decreases.

For a configuration Go board (19x19) with up to more than 350 stones, the measuring time is much longer. Suppose that 15 tags need to be detected per antenna (total 281 tags). The number of anticollision strokes that must be performed per antenna is expected to be approximately 4:
1x with 10 slots with tags (55 ms), 3x with 4 responding tags (3 x 32.4 ms=130 ms), total per antenna approximately 185 ms.
Total measurement time with 38 antennas: 38 x 185 = 7030 ms, so approximately 7 seconds.

For the TikTegel configuration with 8 tags and 50% overlap of the antennas, the measurement time is:
Situation 1: No row or column with 2 tags:
16 x Inventory (1 slot) with 0 tags: 99 ms.
4 x Inventory (1 slot) with 1 tag: 49.2 ms.
All tags are read 4 twice due to the overlap
Total 99 + (4 x 49.2) = 296 ms.

Measurement and development for a Scrabble board
The following times were measured (min and max determined from approximately 5 measurements, during which different tags were placed on the antenna):

[6] Rough determination of the chance of collision:
With 2 tags in an antenna, the chance of no collision within the 16-slot inventory is 15/16. With 3 tags: 15/16 * 14/16. With 4 tags: chance of collision is 33%. With 5 tags the chance of no collision is only 50%.

Number of tags on an antenna	Minimum measured time for INV_AC (ms)	Maximum measured time for INV_AC (ms)	Minimum time per tag	Maximum time per tag
0	18 [7].	19	-	-
1	28	30	28	30
2	38	97	19	49
4	39	88	10	22
8	81	106	10	13
15	162	186	11	12

Now fill the board randomly with 100 tags. And reading an Anticollision with UID per row and column and on both racks (1 block = minimum) gives a total read time of 2.4-2.6 seconds. (a tag is always read twice, so the average reading time per tag is 12.5 ms.)
The method I implemented gives a read time of 450 ms for the same configuration (which is also the maximum scan time!). An acceleration by a factor of 5.5.

Proposed new method

A new coding method is developed to significantly increase the detection speed for the specific usage situation of game boards.

The new method: BitwiseID

Basic idea for the new method: To identify a tag, not the UID is used, but a specific unique pattern stored in the R/W memory of each tag.
The information can be read with a ReadBlock or ReadMultipleBlocks command. In response to this command, a tag sends a number of blocks (4 bytes, 32 bits) of data from the tag's memory.
Now, when this command is executed by the reader without addressing a tag, all tags within the antenna range respond simultaneously. All bits from all tags arrive at the same time. Normally that is undesirable. However, the protocols of the 15693 standard provide for the situation where multiple tags respond simultaneously, and the protocol is able to determine per bit whether the data sent by those tags is identical or not identical.
(footnote: See specification of Reader IC CLRC632, section 10.5.2.4 ColPos register: the reader can detect on which bit an unequal value has been received). This information is also used in the anticollision protocols.
It turns out that when bits collide, the received result is the logical bitwise “OR” of all sent data. Where normally the colliding information becomes useless, now by a smart coding of the data, the collision is of no damage.
If it is ensured that only one tag per bit position is allowed to send out a bit with the value "1", and the other tags have a value "0" for that bit, this “1” bit can be recognized in the response.

[7] This can be accelerated somewhat by first issuing an Inventory command to test whether the antenna is empty. This test lasts 6 ms. If the antenna is not empty, the Anticollision process can still be carried out. The downside is that in all other situations an additional 20 ms is added!

Example:
The tags x, y and z have been programmed with the values 1, 2 and 4 respectively at a certain memory position. Now this memory position of all tags is requested with a ReadBlock command.
Tag x sends the bit stream 00000001 (bit coding of 1)
Tag y sends the bit stream 00000010 (bit coding of 2)
Tag z sends the bit stream 00001000 (bit coding of 8)
This results in a read bitstream of 00001011.
From this it can easily be concluded that tag x, tag y and tag z have been activated by the antenna and have sent their data.
For example, if tag y is *not* present in the antenna field, this results in a read bit stream of 00001001.
With one read operation, all three tags were detected and identified!

When this method is used for a Go board with 400 tiles, i.e. 400 bits, a bit stream of 400 bits (50 bytes) is required. At a bit rate of 25 Kbit/s the time required is 16 ms. Reading 19 row and 19 column antennas will take approximately 38*16 = 608 ms. This is a very useful scan time.

Area of use and preconditions

The following preconditions apply to the new method:

A: A limited number of tags are used per system. These tags are announced in advance of the setup and initialized for this setup. This can be done in different ways:

Option 1: All tags used in a setup are all present on the game board at the start of a session and static for a few seconds. During this time the tags are inventoried and initialized. After this time, no tags other than these will appear on the game board during a session.

Option 2: Tags used in a setup have all been made known to the game board before a session, by placing them on the game board as in option 1 or initialized during production of the measuring system. The properties of the tags are then stored in non-volatile memory of the game board. From that moment on, these tags can be used freely on the game board in arbitrary usage situations (e.g. starting from power-up an empty board, then placing random tags from the announced tags.

Option 3: Random tags can be used on the game board in random usage situations. Tags that have not previously been detected on the game board are registered upon initial placement and added to the setup's list of known tags. This registration takes approximately 50 to 100 ms seconds per tag. The movement or removal of this tag can then be detected based on speed. See “Appendix 5: Elaboration on initialization of tags” for an elaboration of the above proposals.

The following applies to all systems: The behavior of all tags in a system must be identical in timing. This behavior is expected, it is also the base of the conventional Anticollision procedures. Nevertheless, tests with positive results were carried out with tags equipped with Standard Label IC SL2 ICS20 from Philips/NXP.

Further elaboration

A number of extensions and refinements have been developed.

Avoidance/detection of system mixing

It is a risk if in a system/on a board unexpectedly a tag or multiple tags are used that come from another system/board. Then it could easily happen that two tags with the same bit coding are read simultaneously. These tags cannot be distinguished by any means, during the normal reading process.
A solution is to give a system's tags a system-unique identification. This identification must then be read simultaneously with the bit coding. An object from another system (other system identification) must be distinguishable in this group of bits.

A proposed encoding method is to have the system identifier be a group of bits where a prescribed number of bits is “1” and a prescribed number of bits is “0”.
For example: in the 16-bit field a coding is used where the system identification consists of a bit pattern in which exactly 8 bits have the value “1”, and therefore also 8 bits have the value “0”.
When these bit patterns are received in parallel during a read action on an antenna, the result is a logical “OR” on this. If all tags sent the same bit pattern, the result is exactly this bit pattern with exactly 8 bits “1” and 8 bits “0”. If one or more tags send a different bit pattern, the result is a pattern with more than 8 bits with the value “1”.
So, a tag from a foreign system can always be detected because the logical “OR” of the two different identification values results in more than 8 bits as “1”. More than 12,000 system identifiers can be encoded in a 16-bit identification pattern.

Recoding when detecting tag from another set

When a tag is detected that has a system identifier different from the actual system, a procedure must be started to register the foreign tag within the coding of the system. This is done by stepping back to classic anticollision reading and verifying the system code and the tag coding and adapt this, so it fits with the current system.

Resolve interrupted write operation

To rewrite a row or column value in a tag, a write operation must be performed. Such a write operation takes some time, of the order of 10 ms. During this time the tag must remain in the RFID field. There is a risk that with moving tags the write operation is not completely executed because during the write operation, the tag moves outside the energy field. This has also been tested in practice: with moving tags it often happens (5 to 10%) that a write operation fails. In more than 90% of failed writes, the data remained unchanged. Sometimes (in roughly 10% of failed writes) the data in the block being written becomes zero. This is a problem: The tag is then no longer seen because the value zero cannot be distinguished among the data of other tags.

Two solutions were found to solve this problem.
The first is to make two instances of the data block in the tag memory that must be rewritten. Then when a block must be written with new information, this is first executed on the first instance. This is executed until successfully written, then the second instance is

written. If not successful, then at a next occurrence, the executed block was found zero or unchanged. The original data is then always found in the second instance.[8]
The second solution is used for the bit-ID array. A bit variable is defined in a block that indicates that an ID writing is ongoing. If this is successfully set, then the writing of the ID array is started and verified. If indeed correct, the bit variable is cleared.

Resulting measuring speed

With the BitwiseID method, a Go board with 38 antennas and 400 tags can be scanned in approximately 38 x 16 ms = 608 ms
With the BitwiseXY method, a Go board with 38 antennas and 400 tags can be scanned in approximately 38 x 8 ms = 304 ms, with 56 ms per movement. additional measuring time is created.

Simultaneously coding the role of the piece in bitwise array

There can also be room in the data field for each tag to record the role of each tag. In chess there are 12 roles: pawn, rook, knight, bishop, queen and king, each for black and white. These can be encoded in 4 bits (0000 stands for “no tag”). For 34 pieces there is a data length of 136 bits. With 16 bits for coding the system, 152 bits are needed. These can be done in 5.4 ms. sent by the tags.
For Go, 1 bit is sufficient to represent the role of the piece: black or white. 200 bricks require 400 bits.
Co-coding the role of a piece is not necessary in most situations because the role has already been read at BitwiseID or BitwiseXY and is stored in a directly accessible table in the microcontroller

Simple readout with semi-unique numbering of tags

For some applications it is undesirable to perform full set management. An example is a system for registering Scrabble games: Before the letter tiles are placed on the board, they are first placed on the racks of the players. If such a rack is a separate measuring system, it is undesirable to perform set management here. It is also not necessary, because the measurement on the rack can be performed much faster.
There are two options:
A: Single antenna

[8] Further discussion: When scanning an antenna, all four values are requested, in 4 blocks . This costs 3 ms more per antenna compared to requesting 2 block. If the values of the sets do not match, there has been a previous write failure. If the sets are the same but do not correspond to the row/column position, a new value for the coordinates must be written or written after all.
In both situations, one set is first changed (the set whose value is zero or does not correspond to the row/column position). Then this value is read back. If the value is read back correctly, i.e. is written correctly, the second set is also written.
If a value is not read back correctly, the tag has been removed or moved, or the write operation otherwise fails. However, one of the coordinate sets still contains non-zero values. Sooner or later the tag will be found again at another antenna, where the values can then be restored. The time for this procedure for a diagonal displacement is approximately 70 ms [8].
This method has been successfully implemented and tested.

The rack consists of one antenna. This way, the tags on the rack can be detected, but not the position. A full anti-collision procedure on this antenna with reading 4 bytes per tag is fast enough: with 7 tags expected to be about 30 ms.
B: Multiple antennas
The rack consists of several overlapping antennas. By reading these in turn, the identity and position of the tags on the rack can be determined. Also in this situation, a complete anti-collision procedure per antenna is sufficiently fast to read the tags: with 8 antennas 240 ms.

The proposed encoding is that in one memory page (4 bytes, 32 bits) of the tags the following information is stored:

- Letter code
- Number of points
- Possibly tag color and/or other information
- A short (16 to 20 bits) sequence number, assigned when the tag is produced.

The sequence number can have an order of magnitude of 64000 (16 bits) to 1 million (20 bits) different values. If these have all been used, the count starts again from the beginning. So tags do not have a guaranteed unique sequence number, but the chance of two or more identical values appearing on this memory page is negligible.
The rack reading system only needs to request this memory page through an Inventory Read Anti-Collision procedure, and can then communicate the letters on the rack with the main system.

Required tables and data structures

For BitwiseID

The realized software for the board's microcontroller has the following tables for managing tags and recording them

FifoReadBuf : 64 bytes containing the read data of a read operation on an antenna
FifoWriteBuf : 20 bytes containing the command to be sent to the tags on an antenna
EE_ID_USE: List of tag IDs used for a system
EE_ID_ROLE: List the role of these tags
EE_ID_AUX: List of other information of these tags

Conclusion

With the described method it is possible to reduce the measurement time of large game boards with many tags from many seconds to well less than a second, using standard tags and standard Reader electronics.

References

[1] 15693 – 2,
[2] 15693 – 3

Appendix 1: Restrictions

Limit

A limitation was found when reading data from many tags at the same time.
It appears that when receiving data from many simultaneously responding tags, detection becomes more unreliable. The problem occurs when a larger number of tags respond in an antenna. In the event of a colliding ZERO and ONE bit, the entire bit time (37.76 us cq. 18.88 us at FAST) a modulation is present. The decoder turns this into ONE bit. The problem probably arises because the modulation strength of the ZERO bits (which are emitted by almost all tags) is much greater than the modulation strength of the ONE bit (which is emitted by 1 tag). This results in a slightly stronger antenna signal being required for a large number (more than 10) tags on an antenna. This can be a problem with applications like Scrabble and Go.
By using 16-slot anti-collision, the number of responding tags is distributed over 16 slots . Collision can still occur within a slot, but the number of responding tags is limited with a good distribution over the 16 slots , and the disruption will then not occur. So this is a possible solution: use 1 page (32 bits) and use the slot number to supplement the identification: 16 tags can be placed on bit 0 as long as the UIDs for those tags fall into different slots . An enormous speed gain is still achieved compared to . the regular detection with anti-collision over the entire UID. The Fast Inventory for 32 bit data field with all 16 slots occupied can be executed within 20 ms.
Theoretically, a scan time is required (with fast inventory) of 6 command bytes (say 3 ms), 16 x 4 bytes fast mode maximum (1 ms per slot...) together maximum under 20 ms.
With 38 antennas (Go board), a full scan takes 760 ms. (max). This is still usable.

Appendix 2: Further acceleration method BitwiseXY

Row and column coding for further acceleration: BitwiseXY

When the number of bits required to identify a tag is large (e.g. well over 350 bits), the overall scanning speed of a system becomes slow due to the long transport time of these

bits. In such a situation, further acceleration is possible by coding the current coordinates of the tags.
The condition for this method is that there may not be more than one tag in each field, each position.

Explanation BitwiseXY

If a tag is located on the field with row 7 and column 12, these coordinates can be written into the tag in two bit fields. Such a bit field, for example, in a 19x19 system, has 19 bits each for the columns and rows. All tags found in row 7 are assigned (only) bit 7 of the row field "1", the rest of these bits become "0". Tags found in column 12 will have bit 12 of the column field made "1", the rest of these bits will be "0". After all tags in a system have been coded in this way, and this data is read from all tags on a row or column according to the method described under BitwiseID, the result (logical "OR") of a row or column antenna will always have the same value. on the row or column coordinate, namely only the bit corresponding to the row or column number with value "1". In the other field (column or row field) the bits are "1" for the column or row field. row numbers on which a tag was found.
Example: for a system of 4 rows and columns, there are three tags on column 3, namely rows 1, 2 and 4. When the column and row fields of the tags located on column 3 are read, the reading result is explained below. The column and row number are encoded in the four bits from left to right.

Example 1:

	Column field	Row field	
Row 1	0010	1000	
Row 2	- - - -	- - - -	(no tag)
Row 3	0010	0010	
Row 4	0010	0001	
	0010	1011	Result of the read operation on column 3 (logical "OR")

This indicates that all tags on column 3 are coded correctly and that on column 3 there are 3 tags, on rows 1, 3 and 4.

If a tag is moved from row 2, column 2 to row 3, column 2, the following is read:
Example 2:

	Column field	Row field	
Row 1	0010	1000	
Row 2	0100	0100	(no tag)
Row 3	0010	0010	
Row 4	0010	0001	
	0110	1111	

If in example 1 the tag of row 4, column 2 has been removed, the following will be read:

Example 3:

	Column field	Row field
Row 1	0010	1000
Row 2	- - - -	- - - - (no tag)
Row 3	0010	0010
Row 4	- - - -	- - - - (no tag)
	0010	1010

If a tag is moved, this can be determined immediately from the read values of these fields, by comparing them with the previously read values. If a tag is encountered where the column or row field number does not correspond to the number of the reading column or row, it should be identified and then the column and/or row numbers correctly written into the tag.
This method has been successfully implemented.
Identifying a moved tag and adjusting a row or column field of that tag takes less than 35 ms. seized [9].

Appendix 3: Literature and relevant patents

Applications

Roulette application :

http://blog.atlasrfidstore.com/rfid-board-games-casino
http://www.casinojournal.com/ext/resources/White-Papers/Localization-System-for-Roulette-and-Other-Table-Games-Casino-Journal.pdf

Scrabble:

http://www.engadget.com/2012/11/15/scrabble-board-rfid/

Chess patent:

http://www.google.com/patents/US7791483

Wargame :

http://www.vs.inf.ethz.ch/publ/papers/hinske-pg07-rfidtabletop.pdf

Backgammon

Patent Garal :

[9] If a tag is found with a different row or column value, the procedure is as follows:
a) read the Bitwise ID of the tags on the antenna (25 ms)
b) from a static table, find the UID of the tag that has been moved.
c) Write the new coordinate in the tag with UID addressing (10 ms).

http://worldwide.espacenet.com/publicationDetails/originalDocument?CC=US&NR=2007210517A1&KC=A1&FT=D&ND=4&date=20070913&DB=EPODOC&locale=nl_NL
Expired.

To play chess

Patent DGT: US6168158 and 1009574C2
Saitek patent : US5188368 and US5082286
These patents have expired
Citing patents:
http://worldwide.espacenet.com/publicationDetails/citingDocuments?CC=US&NR=6168158B1&KC=B1&FT=D&ND=3&date=20010102&DB=EPODOC&locale=nl_NL

TikTegel

US20110309970A1 TikTegel detection
WO2009024915A3 plurality of positions
sensing physical properties EP2028507
These patents have expired or been refused (EP2028507).

Other possibly related patents

CN104134053A:
http://worldwide.espacenet.com/publicationDetails/description?CC=CN&NR=104134053A&KC=A&FT=D&ND=3&date=20141105&DB=worldwide.espacenet.com&locale=en_EP

Appendix 4: RFID HF standards documentation

15693 standard

Identification cards — Contactless integrated circuit(s) cards — Vicinity cards
—Part 2: Air interface and initialization
—Part 3: Anti-collision and transmission protocol
ICODE SLI Smart Label IC SL2 ICS20 Functional Specification

14443 standard

Identification cards — Contactless integrated circuit(s) cards — Proximity cards
—Part 2: Radio frequency power and signal interface
—Part 3: Initialization and anticollision
—Part 4: Transmission protocol

Mifare standard

http://www.skyetek.com/docs/m2/mifareclassic.pdf .
Understanding the Requirements of ISO/IEC 14443 for Type B Proximity Contactless Identification Cards, Atmel, Rev. 2056B–RFID–11/05

Appendix 5: Set management - Elaboration on initialization of tags

For the practical use of the above methods (BitwiseID or BitwiseXY) it is highly desirable that neither a user nor a system administrator has a task to keep sets of tags separated, but that the measurement system itself manages the situation of mixing of sets.
There are still preconditions that the maximum number of tags on a surface is limited, namely by the number of bits in BitwiseID that are read per scan, or the number of ID bits that are read at BitwiseXY at the time of a (re)placement .

With BitwiseID, this number of bits is read with each antenna scan, so for example a Scrabble board 30 times per board scan. 128 bit ID will lead to approximately 0.35 second scan time, 512 bits will lead to approximately 1.2 second scan time.
With BitwiseXY, this number of bits is only read when a tag is moved in a column or row (twice with diagonal moves). While with 15+15 rows/columns a board scan without displacements takes approximately 0.35 seconds, a displacement with 128 bits ID will add approximately 50 ms, but with 512 bits 80 ms. BitwiseXY is suitable if many tags from large sets can be found on boards.

Considerations for creating an automatic system

- Detecting and resolving mixing of sets via administration (writing to tags) takes time: this is expected to take about 200 ms. per tag cost
- There will often be setups where the same set of tags are always used, and the chance of mixing is small.
- It is undesirable if there comes a time during a registration when the entire administration of tags/sets has to be changed. Such a procedure could easily take 200 ms x 100 tags = 20 seconds.

Proposal 1: Start with old SetID

A table has been created in non-volatile memory with the ID and role of tags, and the SetID A is stored for which that table applies.
Upon detecting an empty board, the following procedure begins:

1. As soon as a first tag is detected, its SetID is compared with the SetID A used in the previous session (for the empty board)

2. a. If these two match, this SetID A is used for this session, and the existing table is used

   b. If these two do NOT match, a new SetID B is generated (Random?) and assigned for this session. The table is cleared
   From this moment on, every newly detected tag (including the first) is described with the new SetID B, an entry is made in the table at the index of the ID read from the tag and the ID and role are filled in the table. A tag's ID will only be reassigned and written if the new tag turns out to have an ID that already exists in the table. There is a small chance that tags will be placed that happen to have this SetID B. When using 31 bits for the SetID , with 15 bits 1, the number of variations is 2.8*10^8, so 280 million. We will just ignore this situation for a moment. Should the situation of two

identical tags (setID and ID) (detectable when finding two identical IDs at two positions) then this new tag needs to be recoded, or all of them?
All better, with new SetID C. There is a good chance that more of this SetID B will be installed. Joint recoding can be done with one instruction for the entire antenna (writeblock without UID).
However, the last tag placed will also receive this SetID C, but the ID has not yet been matched with the table. So different order: First recode the ID of the tag where a duplicate has been found, then all SetIDs recode to SetID C.
The problem remains: If two SetID B tags are placed at the same time, they are both recoded and there remains a chance of duplication .
So: When recoding tags, do this one by one (via UID), which takes a lot of time, or before recoding a row or column, first check whether there is no duplication on that row or column...

Why does a new SetID need to be generated?
Suppose the SetID A of the first tag is used. The first 30 tags all appear to have this SetID A, and then ten tags are added from a series with SetID B. These are added in empty places in the table, and are written SetID A. In fact, new tags are created with SetID A. If tags are added later from the old set A, they may have the same ID as the newly created tags in set A. These are then no longer distinguishable, which is undesirable and causes problems if two lie on the same XY field, or three on the corners of a rectangle.

3. a. The existing table is used for the session. If a tag is placed that has a different SetID , it is added to the table, in an empty spot, and gets a new SetID + ID.

   it may now happen that the table becomes full!
   For example, if after a number of placements of tags with SetID A, a large number of tags are placed with SetID not equal to A, and the table had 100 filled places and 28 unfilled places, a problem arises.
   If table positions are then reused for which the tag is not yet on the board, so for example a new tag is assigned SetID A and ID 10 (reuse), and later the tag that has not yet been placed is placed with SetID A and ID 10, then these are not distinguishable. What to do?
   a1. Before the table becomes full, re-encode all tags to a new SetID C? This can be done for all tags on an antenna simultaneously (WriteSingleBlock without UID info, because a copy of SetID is also needed, this costs 20 ms per antenna, with 30 antennas 600 ms). Afterwards, table entries that have not yet been used can be filled with new tags.

When enabling a board with a number of tags on the board, the following procedure is used:

- First all rows and columns are read. It counts how many rows and columns the SetID matches with the stored SetID , and how many do not match. The following is chosen from the outcome:
  * Create a new table, reassign all tags an ID and SetID . *
  Recode the less found SetID tags and give
  new ID and SetID . This requires a procedure to assign a
  SetID and ID tag by tag.

Proposal 2: More secure procedure, always start with new SetID

Proposal 1 poses a few problems: If a table becomes full, all tags on the board must receive a new SetID . This is a time-consuming and risky procedure: recoding the tags unaddressed with one
WriteBlock runs the risk that tags with a conflicting ID that have been added in the meantime will still be included in the set. A solution is to ALWAYS generate a new random SetID from Powerup or empty board and to always give tags that are present or placed this SetID . The treatment time for this procedure is spread over the session, there are never seconds-long interruptions in scanning.
For the Scrabble application it is very appropriate, because all tags first appear on the racks in a non-dynamic phase of the game. The recoding is carried out there. The manipulations on the board or on the rack (with position-distinguishing racks) are then always quickly shown.

Detailing procedure:
Upon power-up or empty board + empty racks (= new session), a random SessionID is generated. An improved method is to only generate a new SessionID after an empty board + empty racks if the first tag placed does not match the last used SessionID . This has the advantage that in situations where the same set is always used on a system, a new SessionID does not have to be generated each time. Only if apparently a different set is used.
The UID-ID-Function table from the previous session is still available in non-volatile memory.
General procedure:
With each antenna readout, it is always checked whether the SetIDs correspond to the SessionID , i.e. whether the read (combined value of) SetID corresponds to SessionID .
If this is not the case, an Anticollision_Read procedure reads the UID, Role, SetID and ID data of all tags on the antenna and stores it in a table whose SetID must be adjusted. The data per tag is compared with the Set table in non-volatile memory.
AddedInSet array that indicates for each item in the Set table whether that tag has been added and therefore is or has been present since the start of the tag.
Pseudo code:
If SetID (antenna) is different from SessionID
AnticollisionRead by Role, SetID , ID.
    For all tags with incorrect SetID (or InProgress true):
        Compare ID with Set table: if index ID contains the same UID, and AddedInSet [ID] is false , put new UID in table and write new SetID in tag. Done
        AddedInSet [ID] is true (so tag with that ID was already in the set), then ID must be assigned new and SetID written

Note that the SetID must also have a copy in the tag: If a write operation fails due to removing the tag from the field while writing, a value 0 may be created on the block. Therefore, just like with BitwiseXY , add a SetIDcopy .

New Assign ID : Find the first unassigned AddedInSet [] entry and use this ID [10]. To assign the ID to the set, the following steps must be performed:

10

a. Mark the tag as “pending” by writing the new SetID value in the SetID or SetIDcopy block, creating an assigned bit InProgress 1. In the same block, ID can also be written in hexadecimal form (9 bits, also for Go), if 22 bits are reserved for SetID .
b. Check step a by reading back. Then write the other block of SetID or SetIDcopy the new SetID and ID value but without the bit InProgress 1.
c. Check that both SetID blocks are OK. Then write new blocks in the BitwiseID field: Clear the old bit, write the new bit.
d. Check BitwiseID field and if OK, clear the InProgress bit from step 1.

Revised Memory Map for Tags: See Spreadsheet

set management method has been implemented. Registration appears to take approximately 85 ms per tag. (Jan. 2017)
See Software Guide Bitwise version 1 for an explanation of the approach + all functions in the implemented software

Outdated views on set management

A limited number of tags are used per setup. These tags are announced in advance of the setup and initialized for this setup. This can be done in various ways:

Option 1: All tags used in a setup are all present on the game board at the start of a session and static for a few seconds. During this time the tags are inventoried and initialized. After this time, no tags other than these will appear on the game board during a session.

Option 2: Tags used in a setup have all been made known to the game board before a session, by placing them on the game board as in option 1 or initialized during production of the measuring system. The properties of the tags are then stored in non-volatile memory of the game board. From that moment on, these tags can be used freely on the game board in arbitrary usage situations (e.g. starting from power-up an empty board, then placing random tags from the announced tags.

Consequence: If this ID appears from the previous session, it must also be given a new ID. If the set appears in exactly the same order, they should all get a new ID. But that's not likely. For example, a procedure to start searching from the last ID has the same statistic.

Option 3: Random tags can be used on the game board in random usage situations. The measuring system regularly performs a (slower) anti-collision detection method. Any new tags are then initialized. To this end, detection of tags is delayed occasionally, say every 5 seconds, for a short time (0.5 sec). The detection of the placement of a new tag then takes some time [11]. The movement or removal of this tag can then be detected based on speed.

11

Depending on the number of tags on a board. To be investigated. It might not be so bad.